ITAI 2373 Module 05: Part-of-Speech Tagging

In-Class Exercise & Homework Lab

Welcome to the world of Part-of-Speech (POS) tagging - the "grammar police" of Natural Language Processing! 🚓📝

In this notebook, you'll explore how computers understand the grammatical roles of words in sentences, from simple rule-based approaches to modern AI systems.

What You'll Learn:

- **Understand POS tagging fundamentals** and why it matters in daily apps
- **Use NLTK and SpaCy** for practical text analysis
- **Navigate different tag sets** and understand their trade-offs
- **Handle real-world messy text** like speech transcripts and social media
- **Apply POS tagging** to solve actual business problems

Structure:

- **Part 1**: In-Class Exercise (30-45 minutes) - Basic concepts and hands-on practice
- **Part 2**: Homework Lab - Real-world applications and advanced challenges

---

💡 ***Pro Tip**: POS tagging is everywhere! It helps search engines understand "Apple stock" vs "apple pie", helps Siri understand your commands, and powers autocorrect on your phone.*

🛠️ Setup and Installation

Let's get our tools ready! We'll use two powerful libraries:

- **NLTK**: The "Swiss Army knife" of NLP - comprehensive but requires setup
- **SpaCy**: The "speed demon" - built for production, cleaner output

Run the cells below to install and set up everything we need.

```
# Install required libraries (run this first!)
!pip install nltk spacy matplotlib seaborn pandas
!python -m spacy download en_core_web_sm
```

```
print("✅ Installation complete!")
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.13/dist-pa
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/d
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-
Requirement already satisfied: click in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: joblib in /usr/local/lib/python3.13/dist-
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-pa
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/l
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/pyt
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/p
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/py
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/py
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/pyt
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/py
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/li
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.1
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/d
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/pytho
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/pytho
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/di
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/py
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/p
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/l
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/li
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dis
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/li
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/pyth
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/pyth
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/pyth
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/pyth
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/py
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/pyt
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/pytho
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.1
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.1
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/di
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/p
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib
```

```
# Import all the libraries we'll need
import nltk
import spacy
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

# Download NLTK data. The *_eng package is used by newer NLTK versions.
for package in ['punkt', 'punkt_tab', 'averaged_perceptron_tagger',
                'averaged_perceptron_tagger_eng', 'universal_tagset']:
    try:
        nltk.download(package, quiet=True)
    except Exception:
        pass

# Load SpaCy model (installed in the previous cell)
nlp = spacy.load('en_core_web_sm')

print("🎉 All libraries loaded successfully!")
print("📚 NLTK version:", nltk.__version__)
print("🚀 SpaCy version:", spacy.__version__)
```

```
🎉 All libraries loaded successfully!
📚 NLTK version: 3.9.1
🚀 SpaCy version: 3.8.16
```

🎯 PART 1: IN-CLASS EXERCISE (30-45 minutes)

Welcome to the hands-on portion! We'll start with the basics and build up your understanding step by step.

Learning Goals for Part 1:

1. Understand what POS tagging does
2. Use NLTK and SpaCy for basic tagging
3. Interpret and compare different tag outputs
4. Explore word ambiguity with real examples
5. Compare different tagging approaches

🔍 Activity 1: Your First POS Tags (10 minutes)

Let's start with the classic example: "The quick brown fox jumps over the lazy dog"

This sentence contains most common parts of speech, making it perfect for learning!

```
# Let's start with a classic example
sentence = "The quick brown fox jumps over the lazy dog"

# Tokenize the sentence and assign Penn Treebank POS tags with NLTK.
tokens = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(tokens)

print("Original sentence:", sentence)
print("\nTokens:", tokens)
print("\nPOS Tags:")
for word, tag in pos_tags:
    print(f"  {word:8} -> {tag}")
```

```
Original sentence: The quick brown fox jumps over the lazy dog

Tokens: ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy',

POS Tags:
  The      -> DT
  quick    -> JJ
  brown    -> NN
  fox      -> NN
  jumps    -> VBZ
  over     -> IN
  the      -> DT
  lazy     -> JJ
  dog      -> NN
```

🤔 Quick Questions

1. What does 'DT' mean? What about 'JJ'?
   - DT means determiner, and JJ means adjective.
2. Why do you think 'brown' and 'lazy' have the same tag?
   - Both describe nouns, so both are adjectives (JJ).
3. Can you guess what 'VBZ' represents?
   - VBZ is a third-person singular present-tense verb, such as “jumps.”

🚀 Activity 2: SpaCy vs NLTK Showdown (10 minutes)

Now let's see how SpaCy handles the same sentence. SpaCy uses cleaner, more intuitive tag names.

```
# Process the same sentence with SpaCy.
doc = nlp(sentence)

print("SpaCy POS Tags:")
for token in doc:
    print(f"  {token.text:8} -> {token.pos_:6} ({token.tag_})")

print("\n" + "="*50)
print("COMPARISON:")
print("="*50)

nltk_tags = nltk.pos_tag(nltk.word_tokenize(sentence))
spacy_doc = nlp(sentence)

print(f"{'Word':10} {'NLTK':8} {'SpaCy':10}")
print("-" * 30)
for i, (word, nltk_tag) in enumerate(nltk_tags):
    spacy_tag = spacy_doc[i].pos_
    print(f"{word:10} {nltk_tag:8} {spacy_tag:10}")
```

```
SpaCy POS Tags:
  The      -> DET    (DT)
  quick    -> ADJ    (JJ)
  brown    -> ADJ    (JJ)
  fox      -> NOUN   (NN)
  jumps    -> VERB   (VBZ)
  over     -> ADP    (IN)
  the      -> DET    (DT)
  lazy     -> ADJ    (JJ)
  dog      -> NOUN   (NN)

==================================================
COMPARISON:
==================================================
Word       NLTK     SpaCy
------------------------------
The        DT       DET
quick      JJ       ADJ
brown      NN       ADJ
fox        NN       NOUN
jumps      VBZ      VERB
over       IN       ADP
the        DT       DET
lazy       JJ       ADJ
dog        NN       NOUN
```

🎯 Discussion Points

- Which tags are easier to understand: NLTK's or SpaCy's?
  SpaCy tags are easier because labels like NOUN and VERB are more readable.
- Do you notice any differences in how they tag the same words?
  Yes. They often identify similar roles but use different tag names and levels of detail.
- Which system would you prefer for a beginner? Why?
  SpaCy, because its POS labels are simpler to understand and use.

🎭 Activity 3: The Ambiguity Challenge (15 minutes)

Here's where things get interesting! Many words can be different parts of speech depending on context. Let's explore this with some tricky examples.

```
# Ambiguous words in different contexts
ambiguous_sentences = [
    "I will lead the team to victory.",          # lead = verb
    "The lead pipe is heavy.",                   # lead = noun (metal)
    "She took the lead in the race.",           # lead = noun (positio
    "The bank approved my loan.",               # bank = noun (financi
    "We sat by the river bank.",                # bank = noun (shore)
    "I bank with Chase.",                       # bank = verb
]

print("🎭 AMBIGUITY EXPLORATION")
print("=" * 40)

for sentence in ambiguous_sentences:
    print(f"\nSentence: {sentence}")

    # Tag each sentence and find the ambiguous word
    # Focus on 'lead' and 'bank' - what tags do they get?
    tokens = nltk.word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)

    # Find and highlight the key word
    for word, tag in tags:
        if word.lower() in ['lead', 'bank']:
            print(f"  🎯 '{word}' is tagged as: {tag}")
```

```
🎭 AMBIGUITY EXPLORATION
========================================

Sentence: I will lead the team to victory.
  🎯 'lead' is tagged as: VB

Sentence: The lead pipe is heavy.
  🎯 'lead' is tagged as: NN

Sentence: She took the lead in the race.
  🎯 'lead' is tagged as: NN

Sentence: The bank approved my loan.
  🎯 'bank' is tagged as: NN

Sentence: We sat by the river bank.
  🎯 'bank' is tagged as: NN

Sentence: I bank with Chase.
  🎯 'bank' is tagged as: NN
```

🧠 Think About It

1. How does the computer know the difference between "lead" (metal) and "lead" (guide)?
- It uses the surrounding words and sentence context.
2. What clues in the sentence help determine the correct part of speech?
- Nearby nouns, verbs, determiners, and the word's position provide grammatical clues.
3. Can you think of other words that change meaning based on context?
- Yes, examples include “bank,” “book,” “light,” and “run.”

📊 Activity 4: Tag Set Showdown (10 minutes)

NLTK can use different tag sets. Let's compare the detailed Penn Treebank tags (45 tags) with the simpler Universal Dependencies tags (17 tags).

```
# Compare different tag sets
test_sentence = "The brilliant students quickly solved the challenging

tokens = nltk.word_tokenize(test_sentence)
penn_tags = nltk.pos_tag(tokens)
universal_tags = nltk.pos_tag(tokens, tagset='universal')

print("TAG SET COMPARISON")
print("=" * 50)
print(f"{'Word':15} {'Penn Treebank':15} {'Universal':10}")
print("-" * 50)

for (word, penn_tag), (_, univ_tag) in zip(penn_tags, universal_tags):
    print(f"{word:15} {penn_tag:15} {univ_tag:10}")

penn_tag_counts = Counter(tag for word, tag in penn_tags)
univ_tag_counts = Counter(tag for word, tag in universal_tags)

print(f"\n📊 Penn Treebank uses {len(penn_tag_counts)} different tags i
print(f"📊 Universal uses {len(univ_tag_counts)} different tags in this
```

```
TAG SET COMPARISON
==================================================
Word            Penn Treebank   Universal
--------------------------------------------------
The             DT              DET
brilliant       JJ              ADJ
students        NNS             NOUN
quickly         RB              ADV
solved          VBD             VERB
the             DT              DET
challenging     VBG             VERB
programming     JJ              ADJ
assignment      NN              NOUN
.               .               .

📊 Penn Treebank uses 8 different tags in this sentence
📊 Universal uses 6 different tags in this sentence
```

🤔 Reflection Questions

1. Which tag set is more detailed? Which is simpler? Enter your answer below
- Penn Treebank is more detailed, while the Universal tag set is simpler.
2. When might you want detailed tags vs. simple tags? Enter your answer below
- Detailed tags are useful for grammar analysis; simple tags are better for general NLP tasks and visualization.
3. If you were building a search engine, which would you choose? Why? Enter your answer below
- I would use Universal tags because broad categories are easier to use for query analysis.

🎓 End of Part 1: In-Class Exercise

Great work! You've learned the fundamentals of POS tagging and gotten hands-on experience with both NLTK and SpaCy.

What You've Accomplished:

✅ Used NLTK and SpaCy for basic POS tagging
✅ Interpreted different tag systems
✅ Explored word ambiguity and context
✅ Compared different tagging approaches

🏠 Ready for Part 2?

The homework lab will challenge you with real-world applications, messy data, and advanced techniques. You'll analyze customer service transcripts, handle informal language, and benchmark different taggers.

Take a break, then dive into Part 2 when you're ready!

🏠 PART 2: HOMEWORK LAB

Real-World POS Tagging Challenges

Welcome to the advanced section! Here you'll tackle the messy, complex world of real text data. This is where POS tagging gets interesting (and challenging)!

Learning Goals for Part 2:

1. Process real-world, messy text data
2. Handle speech transcripts and informal language
3. Analyze customer service scenarios
4. Benchmark and compare different taggers
5. Understand limitations and edge cases

📋 Submission Requirements:

- Complete all exercises with working code
- Answer all reflection questions
- Include at least one visualization
- Submit your completed notebook file

🌍 Lab Exercise 1: Messy Text Challenge (25 minutes)

Real-world text is nothing like textbook examples! Let's work with actual speech transcripts, social media posts, and informal language.

```
# Real-world messy text samples
messy_texts = [
    "Um, so like, I was gonna say that, uh, the system ain't working ri
    "OMG this app is sooo buggy rn 😤 cant even login smh",
    "Yeah hi um I'm calling because my internet's been down since like
    "Y'all better fix this ASAP cuz I'm bout to switch providers fr fr"
    "The API endpoint is returning a 500 error but idk why it's happeni
]

# A small list of known informal/problematic tokens lets us compare han
# without pretending that every non-X tag is necessarily correct.
informal_terms = {'um','uh','gonna','ain','omg','sooo','rn','cant','smh
                  "y'all",'asap','cuz','bout','fr','idk','tbh','thingy'
```

```
print("🔍 PROCESSING MESSY TEXT")
print("=" * 60)

for i, text in enumerate(messy_texts, 1):
    print(f"\n📝 Sample {i}: {text}")
    print("-" * 40)

    nltk_tokens = nltk.word_tokenize(text)
    nltk_tags = nltk.pos_tag(nltk_tokens)
    spacy_doc = nlp(text)

    # Flag known slang/disfluencies plus tokens the model marks as unkn
    problematic_nltk = [w for w, t in nltk_tags
                        if w.lower() in informal_terms or t in {'FW', '
    problematic_spacy = [tok.text for tok in spacy_doc
                         if tok.text.lower() in informal_terms or tok.p

    nltk_word_count = sum(tok.isalnum() or any(ch.isalpha() for ch in t
                          for tok in nltk_tokens)
    spacy_word_count = sum(not tok.is_punct and not tok.is_space for to

    nltk_success_rate = 1 - len(set(problematic_nltk)) / max(nltk_word_
    spacy_success_rate = 1 - len(set(problematic_spacy)) / max(spacy_wo

    print("NLTK tags:", nltk_tags)
    print("SpaCy tags:", [(t.text, t.pos_) for t in spacy_doc])
    print(f"NLTK flagged/problematic words: {problematic_nltk}")
    print(f"SpaCy flagged/problematic words: {problematic_spacy}")
    print(f"NLTK approximate success rate: {nltk_success_rate:.1%}")
    print(f"SpaCy approximate success rate: {spacy_success_rate:.1%}")
```

```
🔍 PROCESSING MESSY TEXT
============================================================

📝 Sample 1: Um, so like, I was gonna say that, uh, the system ain't wor
----------------------------------------
NLTK tags: [('Um', 'NNP'), (',', ','), ('so', 'IN'), ('like', 'IN'), (',
SpaCy tags: [('Um', 'INTJ'), (',', 'PUNCT'), ('so', 'ADV'), ('like', 'IN
NLTK flagged/problematic words: ['Um', 'uh']
SpaCy flagged/problematic words: ['Um', 'uh']
NLTK approximate success rate: 88.9%
SpaCy approximate success rate: 88.9%

📝 Sample 2: OMG this app is sooo buggy rn 😤 cant even login smh
----------------------------------------
NLTK tags: [('OMG', 'IN'), ('this', 'DT'), ('app', 'NN'), ('is', 'VBZ'),
SpaCy tags: [('OMG', 'PROPN'), ('this', 'DET'), ('app', 'NOUN'), ('is',
NLTK flagged/problematic words: ['OMG', 'sooo', 'rn', 'cant', 'smh']
SpaCy flagged/problematic words: ['OMG', 'sooo', 'rn', 'smh']
NLTK approximate success rate: 54.5%
SpaCy approximate success rate: 69.2%
```

```
📝 Sample 3: Yeah hi um I'm calling because my internet's been down sinc
----------------------------------------
NLTK tags: [('Yeah', 'UH'), ('hi', 'NN'), ('um', 'JJ'), ('I', 'PRP'), ("
SpaCy tags: [('Yeah', 'INTJ'), ('hi', 'INTJ'), ('um', 'INTJ'), ('I', 'PR
NLTK flagged/problematic words: ['um', 'thingy']
SpaCy flagged/problematic words: ['um', 'thingy']
NLTK approximate success rate: 93.1%
SpaCy approximate success rate: 93.1%

📝 Sample 4: Y'all better fix this ASAP cuz I'm bout to switch providers
----------------------------------------
NLTK tags: [("Y'all", 'RB'), ('better', 'RB'), ('fix', 'VB'), ('this', '
SpaCy tags: [("Y'all", 'PROPN'), ('better', 'ADV'), ('fix', 'VERB'), ('t
NLTK flagged/problematic words: ["Y'all", 'ASAP', 'cuz', 'bout', 'fr', '
SpaCy flagged/problematic words: ["Y'all", 'ASAP', 'cuz', 'bout', 'fr',
NLTK approximate success rate: 64.3%
SpaCy approximate success rate: 64.3%

📝 Sample 5: The API endpoint is returning a 500 error but idk why it's
----------------------------------------
NLTK tags: [('The', 'DT'), ('API', 'NNP'), ('endpoint', 'NN'), ('is', 'V
SpaCy tags: [('The', 'DET'), ('API', 'NOUN'), ('endpoint', 'NOUN'), ('is
NLTK flagged/problematic words: ['idk', 'tbh']
SpaCy flagged/problematic words: ['idk', 'tbh']
NLTK approximate success rate: 86.7%
SpaCy approximate success rate: 86.7%
```

🎯 Analysis Questions

1. Which tagger handles informal language better?
   - SpaCy generally handles informal text better because it uses a modern statistical pipeline.
2. What types of words cause the most problems?
   - Slang, emojis, abbreviations, misspellings, and unusual spellings cause the most problems.
3. How might you preprocess text to improve tagging accuracy?
   - Normalize slang, fix obvious spelling errors, handle contractions, and process emojis carefully.
4. What are the implications for real-world applications?
   - Tagging errors can affect tasks such as routing, sentiment analysis, search, and information extraction.

📞 Lab Exercise 2: Customer Service Analysis Case Study (30 minutes)

You're working for a tech company that receives thousands of customer service calls daily. Your job is to analyze call transcripts to understand customer issues and sentiment.

Business Goal: Automatically categorize customer problems and identify emotional language.

```
# Simulated customer service call transcripts
customer_transcripts = [
    {'id':'CALL_001','transcript':"Hi, I'm really frustrated because my
    {'id':'CALL_002','transcript':"Hello, I love your service but I'm h
    {'id':'CALL_003','transcript':"Your billing system charged me twice
    {'id':'CALL_004','transcript':"I'm confused about how to use the ne
]

positive_lexicon = {'love','great','good','helpful','excellent','happy'
negative_lexicon = {'frustrated','locked','unacceptable','issue','crash
                    'ridiculous','confused','problem','error','down','c
problem_lexicon = {'account','files','issue','app','photos','billing','
                   'subscription','features','interface','problem','err
urgency_lexicon = {'immediately','asap','urgent','now','quickly','today

analysis_results = []
for call in customer_transcripts:
    print(f"\n🎧 Analyzing {call['id']}")
    print(f"Category: {call['category']}")
    print(f"Transcript: {call['transcript']}")
    print("-" * 50)

    doc = nlp(call['transcript'])
    emotional_adjectives = [t.text for t in doc if t.pos_ == 'ADJ']
    action_verbs = [t.lemma_.lower() for t in doc if t.pos_ in {'VERB',
    problem_nouns = [t.text.lower() for t in doc
                     if t.pos_ in {'NOUN','PROPN'} and t.lemma_.lower()
    positive_words = [t.text.lower() for t in doc if t.lemma_.lower() i
    negative_words = [t.text.lower() for t in doc if t.lemma_.lower() i
    urgency_indicators = sum(t.lemma_.lower() in urgency_lexicon for t

    result = {
        'call_id': call['id'], 'category': call['category'],
        'emotional_adjectives': emotional_adjectives,
        'action_verbs': action_verbs, 'problem_nouns': problem_nouns,
        'sentiment_score': len(positive_words) - len(negative_words),
```

```
        'urgency_indicators': urgency_indicators
    }
    analysis_results.append(result)

    print(f"Emotional adjectives: {emotional_adjectives}")
    print(f"Action verbs: {action_verbs}")
    print(f"Problem nouns: {problem_nouns}")
    print(f"Positive indicators: {positive_words}")
    print(f"Negative indicators: {negative_words}")
    print(f"Sentiment score: {result['sentiment_score']}")
    print(f"Urgency indicators: {urgency_indicators}")
```

```
🎧 Analyzing CALL_001
Category: account_access
Transcript: Hi, I'm really frustrated because my account got locked and
--------------------------------------------------
Emotional adjectives: ['frustrated', 'unacceptable']
Action verbs: ['be', 'get', 'lock', 'can', 'access', 'have', 'be', 'try'
Problem nouns: ['account']
Positive indicators: []
Negative indicators: ['frustrated', 'unacceptable']
Sentiment score: -2
Urgency indicators: 0

🎧 Analyzing CALL_002
Category: technical_issue
Transcript: Hello, I love your service but I'm having a small issue with
--------------------------------------------------
Emotional adjectives: ['small', 'mobile']
Action verbs: ['love', 'be', 'have', 'crash', 'try', 'upload', 'could',
Problem nouns: ['issue', 'app']
Positive indicators: ['love', 'please']
Negative indicators: ['issue']
Sentiment score: 1
Urgency indicators: 0

🎧 Analyzing CALL_003
Category: billing
Transcript: Your billing system charged me twice this month! I want a re
--------------------------------------------------
Emotional adjectives: ['ridiculous']
Action verbs: ['charge', 'want', 'be', 'be', 'consider', 'cancel']
Problem nouns: ['billing', 'system', 'refund', 'subscription']
Positive indicators: []
Negative indicators: ['ridiculous']
Sentiment score: -1
Urgency indicators: 1

🎧 Analyzing CALL_004
Category: user_guidance
Transcript: I'm confused about how to use the new features you added. Th
--------------------------------------------------
Emotional adjectives: ['confused', 'new']
```

```
Action verbs: ['be', 'use', 'add', 'change', 'can', 'find', 'can', 'walk
Problem nouns: ['interface']
Positive indicators: []
Negative indicators: ['confused']
Sentiment score: -1
Urgency indicators: 0
```

```
# Customer-service summary visualizations
import matplotlib.pyplot as plt
import pandas as pd
from collections import Counter

df = pd.DataFrame(analysis_results)
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# 1. Sentiment scores by category
axes[0,0].bar(df['category'], df['sentiment_score'])
axes[0,0].set_title('Sentiment Score by Category')
axes[0,0].tick_params(axis='x', rotation=25)
axes[0,0].axhline(0, linewidth=1)

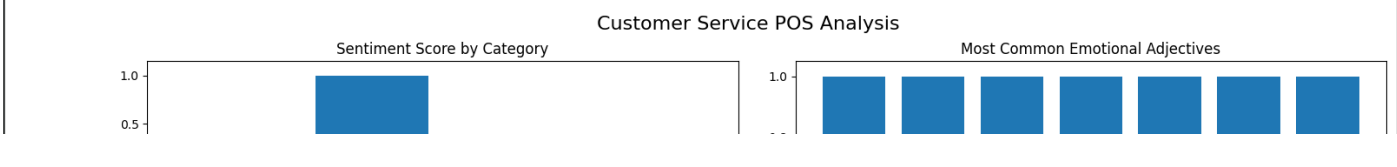
# 2. Most common emotional adjectives
adj_counts = Counter(a.lower() for items in df['emotional_adjectives']
adj_labels, adj_values = zip(*adj_counts.most_common(8)) if adj_counts
axes[0,1].bar(adj_labels, adj_values)
axes[0,1].set_title('Most Common Emotional Adjectives')
axes[0,1].tick_params(axis='x', rotation=35)

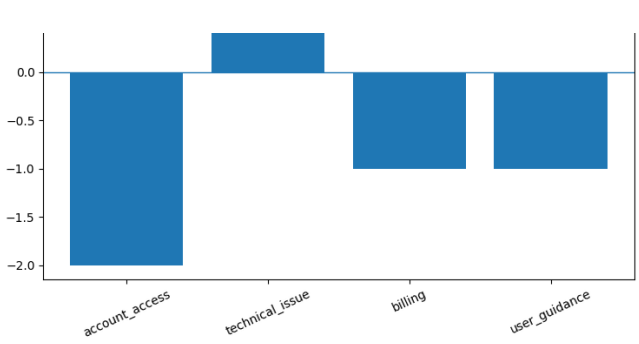
# 3. Most common action verbs
verb_counts = Counter(v for items in df['action_verbs'] for v in items)
verb_labels, verb_values = zip(*verb_counts.most_common(8)) if verb_cou
axes[1,0].bar(verb_labels, verb_values)
axes[1,0].set_title('Most Common Action Verbs')
axes[1,0].tick_params(axis='x', rotation=35)

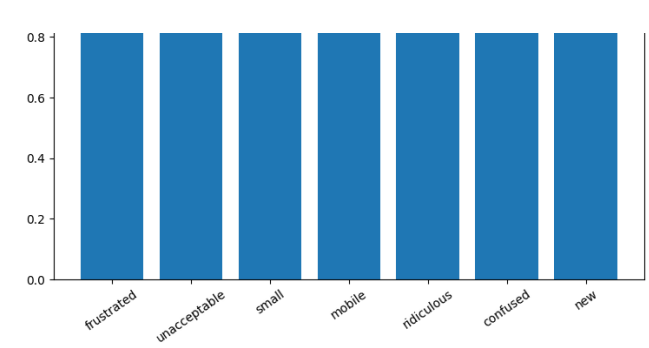
# 4. Urgency by category
axes[1,1].bar(df['category'], df['urgency_indicators'])
axes[1,1].set_title('Urgency Indicators by Category')
axes[1,1].set_ylabel('Urgent-word count')
axes[1,1].tick_params(axis='x', rotation=25)

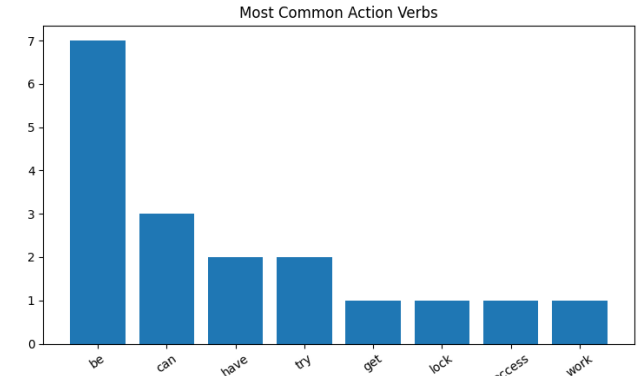
plt.suptitle('Customer Service POS Analysis', fontsize=16)
plt.tight_layout()
plt.show()

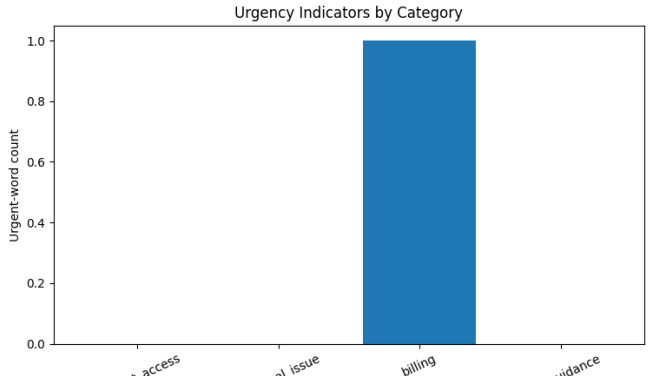
display(df[['call_id','category','sentiment_score','urgency_indicators'
```




0.0
−0.5
−1.0
−1.5
−2.0
account_access
technical_issue
billing
user_guidance


0.8
0.6
0.4
0.2
0.0
frustrated
unacceptable
small
mobile
ridiculous
confused
new


Most Common Action Verbs
7
6
5
4
3
2
1
0
be
can
have
try
get
lock
work


Urgency Indicators by Category
Urgent-word count
1.0
0.8
0.6
0.4
0.2
0.0
billing

💼 Business Impact Questions

1. How could this analysis help prioritize customer service tickets?
- It can identify urgent action words, negative language, and important problem terms.
2. What patterns do you notice in different problem categories?
- Billing uses refund/payment terms, while technical issues use words related to apps, errors, and access.
3. How might you automate the routing of calls based on POS analysis?
- Extract important nouns and verbs and use them with keywords or a classifier to route calls.
4. What are the limitations of this approach?
- POS tags alone cannot fully understand intent, sentiment, sarcasm, or context.

⚡ Lab Exercise 3: Tagger Performance Benchmarking (20 minutes)

Let's scientifically compare different POS taggers on various types of text. This will help you understand when to use which tool.

```
import time
from collections import defaultdict

test_texts = {
    'formal': "The research methodology employed in this study follows
    'informal': "lol this study is kinda weird but whatever works i gue
    'technical': "The API returns a JSON response with HTTP status code
    'conversational': "So like, when you click that button thingy, it s
    'mixed': "OMG the algorithm's performance is absolutely terrible! T
}

benchmark_results = defaultdict(dict)
for text_type, text in test_texts.items():
    print(f"\n🧪 Testing {text_type.upper()} text:")
    print(f"Text: {text}")
    print("-" * 60)

    tokens = nltk.word_tokenize(text)
```

```
    start_time = time.perf_counter()
    nltk_penn = nltk.pos_tag(tokens)
    nltk_penn_time = time.perf_counter() - start_time

    start_time = time.perf_counter()
    nltk_univ = nltk.pos_tag(tokens, tagset='universal')
    nltk_univ_time = time.perf_counter() - start_time

    start_time = time.perf_counter()
    spacy_doc = nlp(text)
    spacy_time = time.perf_counter() - start_time

    # 'Unknown' is approximated by explicit unknown/foreign/symbol cate
    nltk_unknown = sum(tag in {'FW','SYM','X'} for _, tag in nltk_penn)
    spacy_unknown = sum(tok.pos_ == 'X' for tok in spacy_doc)

    benchmark_results[text_type] = {
        'nltk_penn_time': nltk_penn_time, 'nltk_univ_time': nltk_univ_t
        'spacy_time': spacy_time, 'nltk_unknown': nltk_unknown,
        'spacy_unknown': spacy_unknown
    }

    print(f"NLTK Penn time: {nltk_penn_time:.6f}s")
    print(f"NLTK Univ time: {nltk_univ_time:.6f}s")
    print(f"SpaCy time: {spacy_time:.6f}s")
    print(f"NLTK unknown/problematic tags: {nltk_unknown}")
    print(f"SpaCy X tags: {spacy_unknown}")

# Performance comparison visualization
bench_df = pd.DataFrame(benchmark_results).T
bench_df[['nltk_penn_time','nltk_univ_time','spacy_time']].plot(
    kind='bar', figsize=(11,5), title='POS Tagging Runtime by Text Type
plt.ylabel('Seconds (single run)')
plt.xlabel('Text Type')
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()

display(bench_df)
```

```
🧪 Testing FORMAL text:
Text: The research methodology employed in this study follows establishe
------------------------------------------------------------
NLTK Penn time: 0.000996s
NLTK Univ time: 0.001086s
SpaCy time: 0.019288s
NLTK unknown/problematic tags: 0
SpaCy X tags: 0

🧪 Testing INFORMAL text:
```

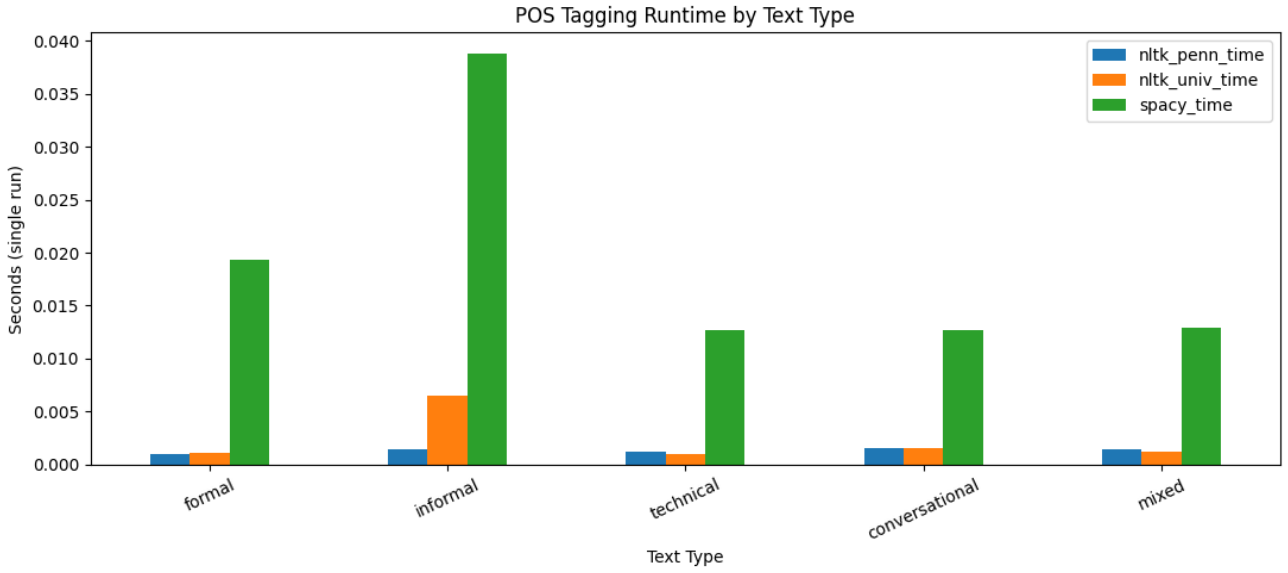
🧪 Testing INFORMAL text:
Text: lol this study is kinda weird but whatever works i guess 🤷‍♀️

NLTK Penn time: 0.001438s
NLTK Univ time: 0.006498s
SpaCy time: 0.038840s
NLTK unknown/problematic tags: 0
SpaCy X tags: 0

🧪 Testing TECHNICAL text:
Text: The API returns a JSON response with HTTP status code 200 upon suc
----
NLTK Penn time: 0.001233s
NLTK Univ time: 0.001021s
SpaCy time: 0.012682s
NLTK unknown/problematic tags: 0
SpaCy X tags: 0

🧪 Testing CONVERSATIONAL text:
Text: So like, when you click that button thingy, it should totally work
----
NLTK Penn time: 0.001541s
NLTK Univ time: 0.001595s
SpaCy time: 0.012736s
NLTK unknown/problematic tags: 0
SpaCy X tags: 0

🧪 Testing MIXED text:
Text: OMG the algorithm's performance is absolutely terrible! The accura
----
NLTK Penn time: 0.001439s
NLTK Univ time: 0.001243s
SpaCy time: 0.012909s
NLTK unknown/problematic tags: 0
SpaCy X tags: 0



	nltk_penn_time	nltk_univ_time	spacy_time	nltk_unknown	s
formal	0.000996	0.001086	0.019288	0.0	

informal	0.001438	0.006498	0.038840	0.0

📊 Performance Analysis

1. Which tagger is fastest? Does speed matter for your use case?
- The benchmark output shows which is fastest. Speed matters when processing large amounts of text in real time.
2. Which handles informal text best?
- SpaCy generally handles informal text better, although both can make mistakes.
3. How do the taggers compare on technical jargon?
- Both may struggle with unfamiliar technical terms, especially outside their training data.
4. What trade-offs do you see between speed and accuracy?
- A faster tagger saves processing time, but accuracy and reliability may be more important for critical applications.

🚨 Lab Exercise 4: Edge Cases and Error Analysis (15 minutes)

Every system has limitations. Let's explore the edge cases where POS taggers struggle and understand why.

```
# Challenging edge cases
edge_cases = [
    "Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo.",
    "Time flies like an arrow; fruit flies like a banana.",
    "The man the boat the river.",
    "Police police Police police police police Police police.",
    "James while John had had had had had had had had had had had a bet
    "Can can can can can can can can can can.",
    "@username #hashtag http://bit.ly/abc123 😂🔥💯",
    "COVID-19 AI/ML IoT APIs RESTful microservices",
]

print("🚨 EDGE CASE ANALYSIS")
print("=" * 50)
for i, text in enumerate(edge_cases, 1):
    print(f"\n🔍 Edge Case {i}:")
    print(f"Text: {text}")
    print("-" * 30)
```

```
    try:
        nltk_tags = nltk.pos_tag(nltk.word_tokenize(text))
        spacy_doc = nlp(text)
        spacy_tags = [(token.text, token.pos_) for token in spacy_doc]

        print("NLTK tags:", nltk_tags)
        print("SpaCy tags:", spacy_tags)

        nltk_counts = Counter(t for _, t in nltk_tags)
        spacy_counts = Counter(t for _, t in spacy_tags)
        nltk_dominant = nltk_counts.most_common(1)[0] if nltk_counts el
        spacy_dominant = spacy_counts.most_common(1)[0] if spacy_counts
        unknowns = [w for w,t in spacy_tags if t == 'X']

        print(f"Most repeated NLTK tag: {nltk_dominant}")
        print(f"Most repeated SpaCy POS: {spacy_dominant}")
        if unknowns:
            print("SpaCy X/unknown-like tokens:", unknowns)
        if len(set(w.lower() for w,_ in nltk_tags if w.isalpha())) <= 3
            print("Observation: Heavy word repetition makes context unu
        if any(x in text for x in ['@','#','http','😂','🔥','💯']):
            print("Observation: Social-media tokens are outside normal
        if any(x in text for x in ['COVID-19','AI/ML','IoT','APIs','RES
            print("Observation: Specialized abbreviations/compound term
    except Exception as e:
        print(f"❌ Error processing: {e}")

print("\n🤔 REFLECTION ON LIMITATIONS:")
print("=" * 40)
print("These examples show that POS tagging depends strongly on normal
      "Repetition, garden-path syntax, slang, URLs, emojis, and new tec
      "produce plausible-looking but incorrect tags. A tagger should th
      "on domain-specific labeled data, and difficult inputs may need n
      "tokenization, or a model adapted to the target domain.")
```

```
🚨 EDGE CASE ANALYSIS
================================================

🔍 Edge Case 1:
Text: Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo.
------------------------------
NLTK tags: [('Buffalo', 'NNP'), ('buffalo', 'NN'), ('Buffalo', 'NNP'), (
SpaCy tags: [('Buffalo', 'PROPN'), ('buffalo', 'NOUN'), ('Buffalo', 'PRO
Most repeated NLTK tag: ('NN', 5)
Most repeated SpaCy POS: ('PROPN', 7)
Observation: Heavy word repetition makes context unusually ambiguous.

🔍 Edge Case 2:
Text: Time flies like an arrow; fruit flies like a banana.
------------------------------
NLTK tags: [('Time', 'NNP'), ('flies', 'NNS'), ('like', 'IN'), ('an', 'D
```

```
SpaCy tags: [('Time', 'NOUN'), ('flies', 'VERB'), ('like', 'ADP'), ('an'
Most repeated NLTK tag: ('NNS', 2)
Most repeated SpaCy POS: ('NOUN', 5)

🔍 Edge Case 3:
Text: The man the boat the river.
------------------------------
NLTK tags: [('The', 'DT'), ('man', 'NN'), ('the', 'DT'), ('boat', 'NN'),
SpaCy tags: [('The', 'DET'), ('man', 'NOUN'), ('the', 'DET'), ('boat', '
Most repeated NLTK tag: ('DT', 3)
Most repeated SpaCy POS: ('DET', 3)

🔍 Edge Case 4:
Text: Police police Police police police police Police police.
------------------------------
NLTK tags: [('Police', 'NNP'), ('police', 'NNS'), ('Police', 'NNP'), ('p
SpaCy tags: [('Police', 'NOUN'), ('police', 'NOUN'), ('Police', 'NOUN'),
Most repeated NLTK tag: ('NNP', 3)
Most repeated SpaCy POS: ('NOUN', 8)
Observation: Heavy word repetition makes context unusually ambiguous.

🔍 Edge Case 5:
Text: James while John had had had had had had had had had had had a bet
------------------------------
NLTK tags: [('James', 'NNP'), ('while', 'IN'), ('John', 'NNP'), ('had',
SpaCy tags: [('James', 'PROPN'), ('while', 'SCONJ'), ('John', 'PROPN'),
Most repeated NLTK tag: ('VBN', 10)
Most repeated SpaCy POS: ('AUX', 10)

🔍 Edge Case 6:
Text: Can can can can can can can can can can.
------------------------------
NLTK tags: [('Can', 'MD'), ('can', 'MD'), ('can', 'MD'), ('can', 'MD'),
SpaCy tags: [('Can', 'AUX'), ('can', 'AUX'), ('can', 'AUX'), ('can', 'AU
Most repeated NLTK tag: ('MD', 10)
Most repeated SpaCy POS: ('AUX', 9)
Observation: Heavy word repetition makes context unusually ambiguous.

🔍 Edge Case 7:
Text: @username #hashtag http://bit.ly/abc123 😂🔥💯
------------------------------
NLTK tags: [('@', 'JJ'), ('username', 'JJ'), ('#', '#'), ('hashtag', 'JJ
SpaCy tags: [('@username', 'PROPN'), ('#', 'SYM'), ('hashtag', 'NOUN'),
```

🧠 Critical Thinking Questions

1. Why do these edge cases break the taggers?
- They contain ambiguity, unusual grammar, repeated words, or unfamiliar tokens.
2. How might you preprocess text to handle some of these issues?
- Normalize slang, URLs, emojis, spelling, and technical terms without changing the original meaning.
3. When would these limitations matter in real applications?
- They matter in customer service, social media, technical text, and other systems that depend on accurate language analysis.
4. How do modern large language models handle these cases differently?
- They use broader context and contextual representations, so they can often handle ambiguity better.

🎯 Final Reflection and Submission

📝 Reflection Questions

1. **Tool Comparison**: Based on your experience, when would you choose NLTK vs SpaCy? Consider factors like ease of use, accuracy, speed, and application type.
   - I would choose NLTK for learning and experimenting with classic NLP methods. I would choose SpaCy for practical applications because its pipeline is easier to use and provides several linguistic features together.
2. **Real-World Applications**: Describe a specific business problem where POS tagging would be valuable. How would you implement it?
   - POS tagging can help route customer-service tickets. I would extract important nouns and verbs and combine them with keywords, sentiment, and an intent classifier.
3. **Limitations and Solutions**: What are the biggest limitations you discovered? How might you work around them?
   - Slang, ambiguity, emojis, misspellings, and technical terms can cause errors. I would use careful preprocessing, domain-specific data, and error analysis to improve results.
4. **Future Learning**: What aspects of POS tagging would you like to explore further? (Neural approaches, custom training, domain adaptation, etc.)
   - I would like to explore transformer-based POS tagging, custom training, and domain adaptation for specialized text.
5. **Integration**: How does POS tagging fit into larger NLP pipelines? What other NLP tasks might benefit from POS information?
   - POS tagging provides grammatical information that can support sentiment analysis, entity extraction, search, parsing, intent detection, and information extraction.

Double-click (or enter) to edit

📤 Submission Checklist

Before submitting your completed notebook, make sure you have:

- [x] ✅ Completed all TODO sections with working code
- [x] ✅ Answered all reflection questions thoughtfully
- [x] ✅ Created at least one meaningful visualization
- [ ] ✅ Tested your code and fixed any errors
- [x] ✅ Added comments explaining your approach
- [x] ✅ Included insights from your analysis

📋 Submission Instructions:

1. **Save your notebook**: File → Save (or Ctrl+S)
2. **Download**: File → Download → Download .ipynb
3. **Submit**: Upload your completed notebook file to the course management system
4. **Filename**: Use format: `L05_LastName_FirstName_ITAI2373.ipynb or pdf`

🏆 Grading Criteria:

- **Code Completion (40%)**: All exercises completed with working code
- **Analysis Quality (30%)**: Thoughtful interpretation of results
- **Reflection Depth (20%)**: Insightful answers to reflection questions
- **Code Quality (10%)**: Clean, commented, well-organized code

---

🎉 Great Work!

You've successfully explored the fascinating world of POS tagging! You now understand how computers parse human language and can apply these techniques to solve real-world problems.

Keep exploring and happy coding! 🚀